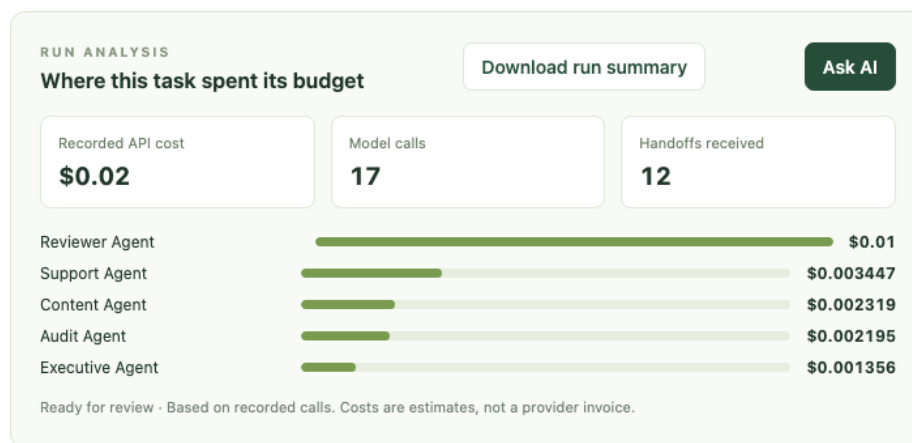


Make every Agent task accountable.

A guided Reins pilot for one existing Python workflow

The first step is observation. See what a task did, what it cost, and whether its result passed your business check. Review the evidence before choosing runtime controls.



Public demo: the recorded cost, calls and handoffs of one multi-Agent task.

What you need

- Python 3.10+, a new virtual environment, and read-only access to the private `catyans/reins` repository.
- One real multi-step task, its existing tests, and a named engineer who can define an accepted result.
- A customer-local storage path; keep provider keys and raw run data in your environment.

Install the pilot version

```
git clone git@github.com:catyans/reins.git /path/to/reins
python -m pip install /path/to/reins
```

Ask your Reins contact for access to the private product repository. This public kit contains the guides, examples and integration skill.

Full, copyable instructions: github.com/catyans/reins-customer-kit. All sample prices and inputs are constructed.

1 / See a whole task, not just its model calls

Wrap one complete business task in `@trace` and configure Reins once before work starts. Observation records what happens without blocking calls. Let your own business check decide whether the result is usable.

```
configure(storage_path="./pilot.duckdb", mode="observe")
@trace(agent_name="supplier_agent", task_type="supplier_lookup")
async def research(source):
    result = await existing_agent(source)
    record_outcome(success=required_fields_pass(result))
    return result
```

Run the original and wrapped task on the same inputs. Compare prompts, outputs and external side effects. Then use `reins compare --database ./pilot.duckdb --task-type supplier_lookup`. A process-local timeline is available by setting `dashboard=True` and opening `127.0.0.1:8765` while the Agent runs.

2 / Connect the calls you want to understand

Your Agent uses	How to connect it
OpenAI Chat Completions or Anthropic Messages	Calls within the traced task are recorded through the supported SDK path.
Native Gemini or a paid tool	Wrap each paid operation with <code>Workflow.call/acall</code> and report its actual cost.
Framework callbacks / OTel	Add context to the record; controlling spend also needs a connected paid-call path.
HTTP proxy	Can be used for experimental observation.

3 / Decide when to add budget rules

After reviewing real runs, agree on a task budget, a reliable upper limit for each paid call, allowed models and tools, and what to keep if a task stops. Try `mode="enforce"` on a small set of *new* tasks. Calls already sent may still be billed.